# INTRODUCTION

## Description:

The **Weather Forecast System Using API** is a web-based application designed to provide users with current and forecasted weather information for a selected location. The main purpose of this project is to make weather information easily available to users through a simple and user-friendly interface. Instead of depending on newspapers, television, or manually searching different websites, users can enter the name of a city and quickly get the required weather details.

The system uses a **Weather API (Application Programming Interface)** to collect real-time weather data from an online weather service. When the user enters a city name, the application sends a request to the weather API. The API processes the request and sends weather information back to the application in a structured format, usually **JSON**. The application then extracts the required information and displays it to the user.

The system can display important weather details such as **city name, current temperature, weather condition, humidity, wind speed, atmospheric pressure, and visibility**. Depending on the API used, the system can also provide forecast information for upcoming days. Weather conditions can be displayed using simple text and weather icons so that users can understand the information easily.

The project can be developed using technologies such as **Python, HTML, CSS, and JavaScript**. Python can be used to handle the backend operations and communicate with the Weather API. HTML is used to create the structure of the webpage, while CSS is used to design and improve the appearance of the application. JavaScript can be used to add interactive features to the system. A framework such as **Flask** can also be used to develop the web application.

One of the main concepts demonstrated by this project is **API integration**. An API allows the application to communicate with an external weather service

without storing all weather data locally. Whenever the user searches for a location, the application can request the latest available weather information from the API. This makes the system more dynamic and useful compared to a system containing only static weather data.

The system also includes basic error handling. If the user enters an invalid city name, the application can display a suitable message. Similarly, if there is an internet connection problem or the weather API is temporarily unavailable, the system can inform the user instead of showing incorrect information.

## Real-world relevance:

Weather information is an important part of everyday life because weather conditions can affect people, transportation, agriculture, travel, businesses, and outdoor activities. Knowing the current and upcoming weather conditions helps people make better decisions and prepare for possible changes in the weather. The **Weather Forecast System Using API** provides a simple way to access updated weather information through the internet.

In the real world, people often need weather information before planning their daily activities. For example, a student may check the weather before going to college, a person may check the forecast before travelling, or a family may check the weather before planning an outdoor activity. By providing weather information in one simple application, the system saves time and makes the information easier to access.

Weather forecasting is also important in **agriculture**. Farmers depend on weather conditions for activities such as sowing seeds, irrigation, harvesting, and protecting crops. Information about rainfall, temperature, and wind conditions can help farmers plan their activities more effectively. Although this project is not designed specifically for agricultural decision-making, it demonstrates how weather data can be accessed and presented to users.

The project is also useful for **travel and tourism**. Before travelling to another city or location, people can check the expected weather conditions and plan their clothing, transportation, and activities accordingly. For example, if heavy rainfall is expected, a traveller may change outdoor plans or carry appropriate equipment.

Weather conditions can also affect **transportation and outdoor events**. Airlines, road transport services, and event organizers need weather information for planning and safety purposes. Heavy rain, strong winds, storms, or extreme temperatures can create difficulties in transportation and outdoor activities. A weather information system can provide useful information that supports better planning.

Another important real-world relevance of this project is **API integration**. Modern websites and applications frequently use APIs to obtain information from external services. For example, applications use APIs for maps, payments, news, social media, currency conversion, and weather information. This project provides practical knowledge about how an application sends a request to an external server, receives data, processes it, and displays the result to the user.

The project also demonstrates the use of **real-time data**. Weather information changes continuously, so users need updated information rather than old stored data.

By connecting the application to a weather API, the system can request the latest available information whenever the user searches for a location. This makes the application more practical for real-world use.

From an educational point of view, this project is important because it helps students understand how different parts of a modern application work together. Students can learn about **web development, API requests, JSON data, frontend design, backend programming, error handling, and internet-based services**. These concepts are commonly used in real-world software development.

The project can also be expanded into a more advanced system. Features such as automatic location detection, multiple-day forecasts, weather alerts, weather maps, air-quality information, sunrise and sunset timings, and mobile notifications can be added in the future. Therefore, the project provides a basic foundation for developing a complete weather application.

Overall, the Weather Forecast System Using API matters because it provides **quick access to useful weather information** while demonstrating a technology that is widely used in modern applications. It connects a simple student-level project with a real-world problem and shows how APIs can be used to create practical, dynamic, and user-friendly software systems.

# OBJECTIVE

The main objective of the **Weather Forecast System Using API** is to develop a simple and user-friendly application that can provide updated weather information for a selected location. The system is designed to make weather information easily accessible to users by connecting the application with an online Weather API. Instead of manually searching for weather information from different sources, the user can enter the name of a city and get the required weather details through the system.

**The following are the major objectives of the project:**

**1. To develop a simple Weather Forecast System:**

The first objective of this project is to develop a simple system that can provide weather information to users. The system should be easy to understand and easy to operate. The user does not require advanced technical knowledge to use the application. By entering a city or location, the user can obtain the available weather information.

**2. To provide current weather information:**

The system aims to provide current weather information for the selected location. Weather information may include current temperature, weather condition, humidity, wind speed, atmospheric pressure, and other available details. This helps users understand the present weather condition of a particular location.

**3. To use Weather API for retrieving weather data:**

An important objective of this project is to learn and implement the use of a **Weather API**. The API allows the application to communicate with an external weather service and retrieve weather information. Instead of creating a large database of weather information manually, the system requests the required data from the Weather API.

**4. To retrieve updated weather data:**

Weather conditions can change frequently. Therefore, another objective is to retrieve updated information whenever the user searches for a location. The application sends a request to the Weather API and receives the latest available weather data. This makes the system more useful than a system based only on fixed or previously stored information.

**5. To provide a user-friendly interface:**

The project aims to create a simple and user-friendly interface. Users should be able to understand the different fields and weather information without difficulty. The application can contain a search box where the user enters the city name and a button to search for the weather information.

**6. To display weather information clearly:**

Another objective is to display the retrieved weather information in a clear and organized manner. Important information such as city name, temperature, humidity, wind speed, and weather condition should be displayed properly. A well-organized display helps users understand the weather information quickly.

**7. To allow users to search different locations:**

The system should allow users to search for weather information for different cities or locations. The user can enter a location name and request its weather information. This makes the application useful for people who want to check the weather of different places.

**8. To understand API request and response:**

This project aims to help students understand how an application communicates with an external server. When the user searches for a city, the application sends a request to the Weather API. The API processes the request and returns a response containing weather data. Understanding this request-and-response process is an important part of modern web application development.

**9. To understand JSON data:**

Most Weather APIs provide their response in **JSON (JavaScript Object Notation)** format. Therefore, another objective of the project is to understand how JSON data is received and processed by the application. The system extracts the required information from the JSON response and displays it on the webpage.

**10. To provide weather forecast information:**

The project also aims to provide forecast information for upcoming periods, depending on the features available in the selected Weather API. Forecast information can help users plan their activities in advance. For example, users can check whether the weather is expected to be sunny, cloudy, or rainy.

**11. To reduce the time required to find weather information:**

Another objective is to save the user's time. Normally, a person may need to search the internet and visit different websites to find weather information. This system provides the required information through a single application. The user only needs to enter the location and request the weather data.

**12. To handle invalid location input:**

The system should be able to handle incorrect or invalid location names. If the user enters a city that cannot be found, the system should display an appropriate message instead of showing incorrect information. This makes the system more reliable and user-friendly.

**13. To handle API and internet errors:**

The project also aims to implement basic error handling. Sometimes the Weather API may not be available, or the user's internet connection may not work properly. In such situations, the application should handle the error and provide a suitable message to the user rather than stopping unexpectedly.

**14. To understand real-time data retrieval:**

One of the important learning objectives of this project is to understand **real-time data retrieval**. The system does not depend only on static information stored in the application. Instead, it communicates with an online service and retrieves

weather information when required. This provides practical knowledge about dynamic applications.

**15. To apply programming concepts in a practical project:**

The project aims to apply programming concepts learned during the course to develop a useful real-world application. Concepts such as variables, functions, conditions, input handling, API requests, data processing, and error handling can be used while developing the system.

## Example:

"To develop a Weather Forecast System that can provide real-time weather information for a selected city using a Weather API. The system will allow users to search for a location and display important details such as temperature, humidity, wind speed, and weather conditions. It aims to provide updated weather information in a simple and user-friendly format."

# PROBLEM STATEMENT

## Problem Identification:

Weather conditions change continuously and can affect our daily activities such as travelling, farming, transportation, outdoor events, and daily planning. People need accurate and updated weather information to make better decisions. However, finding weather information manually can sometimes take time and may require checking different sources.

The **Weather Forecast System Using API** aims to solve this problem by providing weather information through a simple and user-friendly application. The system allows users to enter the name of a city or location and obtain its current weather information. The system connects with an online Weather API to retrieve updated weather data and displays the information to the user.

The system can provide important details such as **temperature, humidity, wind speed, atmospheric pressure, and weather conditions**. Depending on the selected API, it can also provide forecast information for upcoming days. This allows users to get useful weather information in one place without searching through multiple sources.

The system also helps reduce the effort required to obtain weather information. Instead of manually checking different websites, newspapers, or other sources, users can simply enter the required location and get the available weather information through the application.

Therefore, the main problem addressed by this project is the **difficulty and inconvenience of obtaining updated weather information quickly and easily**. The proposed system provides a convenient solution by retrieving weather data through an API and presenting it in a simple format.

## Description:

Before using online weather applications and API-based systems, people commonly depended on different manual methods to obtain weather information. These methods included watching weather reports on television, listening to weather updates on the radio, reading newspapers, or manually searching for weather information on different websites.

In the manual process, a person first needs to find a suitable source of weather information. For example, the user may have to wait for a weather report on television or search through a newspaper. If the user wants weather information for a different city, they may need to search for another source. This process can be inconvenient when quick and updated information is required.

Another manual method is searching for weather information on the internet. The user may need to open a search engine, enter the city name, find a suitable weather website, and then check the required information. If the user wants to compare weather information for multiple locations, the same process may need to be repeated several times.

**The manual process has several limitations:**

**a) Time Consuming**

Manual searching for weather information can take more time, especially when the user needs information for multiple locations. A dedicated weather system can provide the required information much faster.

**b) Lack of Updated Information**

Newspaper and other traditional weather reports may not always provide the latest available information. Weather conditions can change quickly, so updated data is important.

**c) Multiple Sources Required**

Users may need to check different sources to obtain different weather details. This can make the process inconvenient and confusing.

**d) Difficulty in Searching Multiple Locations**

If a user wants to check the weather of several cities, the manual process has to be repeated for each location. An API-based system can make this process easier by allowing users to search different locations through the same application.

**e) Difficult to Access Detailed Information**

Traditional weather reports may not always provide all the information required by the user. Details such as humidity, wind speed, pressure, and other weather parameters may require additional searching.

**f) Human Error**

When information is collected manually from different sources, there is a possibility of misunderstanding or recording incorrect information. An automated system reduces the need for manual collection and display of data.

**g) Less Convenient for Users**

Traditional methods require users to spend time searching or waiting for weather reports. A digital weather forecasting system provides information directly through an application, making the process more convenient.

**h) Limited User Interaction**

Traditional weather reports generally provide information in a fixed format. An API-based system can allow users to search for different cities and obtain information according to their requirements.

**Proposed Solution**

To overcome these limitations, the proposed **Weather Forecast System Using API** provides an automated way of obtaining weather information. The user enters a city or location, and the system sends a request to the Weather API. The API returns the available weather data, and the system processes and displays the required information to the user.

The proposed system reduces the time and effort required to search for weather information manually. It also provides a simple interface through which users can check weather conditions for different locations. Thus, the system provides a more convenient and efficient method of accessing weather information compared with traditional manual methods.

# SCOPE OF THE PROJECT

## Included in the System:

- User can enter a **city or location name**.
- System will retrieve weather data using a **Weather API**.
- System will display **current temperature**.
- System will display **humidity** and **wind speed**.
- System will display the **current weather condition** such as sunny, cloudy, or rainy.
- System can display **weather forecast information** if supported by the API.
- System will provide a **simple and user-friendly interface**.
- System will handle **invalid city names and basic API errors**.
- Weather information will be obtained from an **online API** rather than being manually entered into the system.

## Excluded from the System

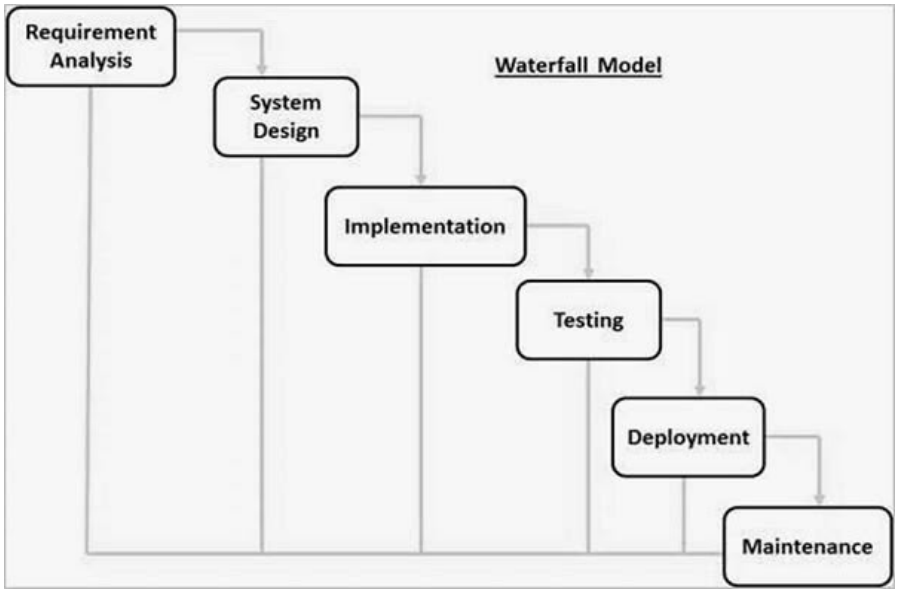
- The system will not **predict weather independently** using its own machine-learning model.
- The system will not collect weather data using **physical weather sensors**.
- Advanced weather analysis and scientific weather prediction are not included.
- The system will not provide **professional meteorological reports**.
- Features such as advanced disaster prediction, satellite weather analysis, and professional weather monitoring are outside the scope of this micro project.
- The accuracy of the weather information depends on the **Weather API and its data source**.

# SOFTWARE ENGINEERING CONCEPTS APPLIED

## Software Development Life Cycle (SDLC):

For the Weather Forecast System Using API, the linear-sequential Waterfall Model of the Software Development Life Cycle (SDLC) was implemented because the project had well-defined, static requirements and a focused development scope. The engineering process moved systematically through six discrete phases: starting with Requirement Analysis to identify essential inputs and core parameters such as temperature, humidity, and wind speed; progressing to System Design where the user interface, DFDs, and UML models were mapped out; executing Implementation using Python, Flask, HTML, CSS, and JavaScript to handle HTTP requests and JSON data parsing; conducting comprehensive Testing (both unit and functional tests) to validate city inputs and verify network error-handling routines; followed by Deployment of the functional web application; and establishing a Maintenance plan to support potential API contract changes and future feature updates.

**Waterfall Model:**



The **Software Development Life Cycle (SDLC)** is a systematic process used for developing software applications. It provides a proper sequence of steps that helps developers to plan, design, develop, test, and maintain a software system. For our **Weather Forecast System Using API**, we followed the **Waterfall Model** of SDLC.

The Waterfall Model is a simple and sequential software development model. In this model, the development process is divided into different phases. Each phase is completed before moving to the next phase. The main phases include **Requirement Analysis, System Design, Implementation, Testing, Deployment, and Maintenance**.

The Water Forecast System was developed by following these phases step by step.

**1.Requirement Analysis:**

In the first phase, the requirements of the system were identified. The main purpose of our project is to provide current weather information to the user. The system requires a weather API to collect real-time weather data. The important information required from the API includes **temperature, humidity, wind speed, weather condition, and location**.

**2.System Design:**

After collecting the requirements, the design of the system was planned. The user interface was designed in a simple and user-friendly way. The system contains a search box where the user can enter a city name. After entering the city name, the application sends a request to the weather API. The API provides the required weather information, which is then displayed on the screen. The design also includes proper arrangement of weather details so that the user can easily understand the information.

**3.Implementation:**

In this phase, the actual coding of the Weather Forecast System was done. The required programming languages and technologies were used to develop the

application. The weather API was integrated with the system to obtain real-time weather data. When the user enters a city name, the application sends a request to the API. The API processes the request and returns weather information. This information is then displayed to the user through the application interface.

**4.Testing:**

After implementation, the system was tested to check whether it works correctly or not. Different city names were entered into the system to check the weather results. The API response was checked to make sure that the correct temperature, humidity, wind speed, and weather condition were displayed. Invalid city names and incorrect inputs were also considered. Testing helped to identify and fix errors in the system. This phase ensured that the application works properly before final use.

**5.Deployment:**

After successful testing, the Weather Forecast System was made ready for use. The required files, API connection, and user interface were properly configured. The system can be run by the user to search for weather information of different locations. Deployment is the stage where the completed software is provided for actual use.

**6.Maintenance:**

The final phase of the Waterfall Model is maintenance. After deployment, the system may require changes or improvements. For example, if the weather API changes its format or stops providing a particular service, the application may need to be updated. New features can also be added in the future, such as a **5-day weather forecast, weather icons, location detection, and temperature unit selection**. Maintenance helps to keep the software working correctly for a longer period.

**Why Waterfall Model Was Used:**

The **Waterfall Model** was selected for this project because it is simple and easy to understand. The Weather Forecast System is a small project with clearly defined requirements. The development process can be divided into different stages, and each stage can be completed before starting the next one. It also makes the project easier to manage and document.

The Waterfall Model helps us to maintain a proper sequence during development. First, we understand the requirements, then design the system, write the code, test the application, and finally deploy it. This systematic approach reduces confusion and helps in finding errors during the testing phase.

Therefore, the **Waterfall SDLC model** was suitable for developing our Weather Forecast System Using API because the project has simple requirements, a limited scope, and a clearly defined development process.

## Requirement Analysis:

Requirement Analysis is an important phase of the **Software Development Life Cycle (SDLC)**. In this phase, we identify and understand the requirements of the **Weather Forecast System Using API**. It helps us to decide what the system should do and how the system should perform.

For this project, the requirements are divided into two main categories:

1. **Functional Requirements**
2. **Non-Functional Requirements**

**1.Functional Requirements:**

Functional requirements describe **what the system must do**. These are the main functions provided by the Weather Forecast System.

**a) User Input:**

The system must provide a search box where the user can enter the **name of a city or location**. The user should be able to easily enter the required location.

**b) Weather API Connection:**

The system must connect with a **Weather API** to get current weather information. The API provides the required weather data from an external weather service.

**c) Fetch Weather Information:**

When the user enters a city name and clicks the search button, the system must send a request to the Weather API. The API response is then received and processed by the system.

**d) Display Temperature:**

The system must display the **current temperature** of the selected city. The temperature should be displayed clearly so that the user can easily understand it.

**e) Display Weather Condition:**

The system must display the current weather condition, such as **Sunny, Cloudy, Rainy, Clear, or Partly Cloudy**, according to the information received from the API.

**f) Display Humidity:**

The system must show the **humidity percentage** of the selected location. This gives the user information about the moisture level in the air.

**g) Display Wind Speed:**

The system must display the **current wind speed** of the selected city. This information is useful for understanding the current wind conditions.

**h) Display Location:**

The system must display the searched **city/location name** along with its weather information. This helps the user identify which location's weather is being displayed.

**i) Error Handling:**

If the user enters an invalid city name or if weather information is not available, the system should display a suitable **error message**. The application should not crash because of incorrect input.

**j) Search Different Locations:**

The user should be able to search for the weather of **different cities** one after another. The system should provide updated information for every new search.

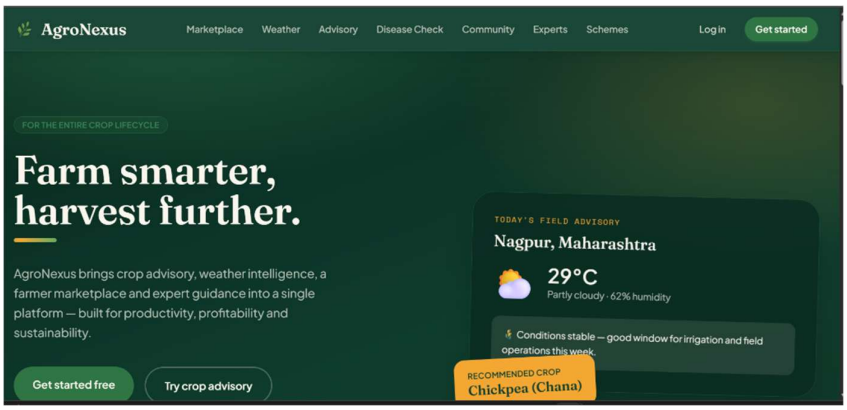



**Figure 9.2: Main User Interface of Weather Forecast System**

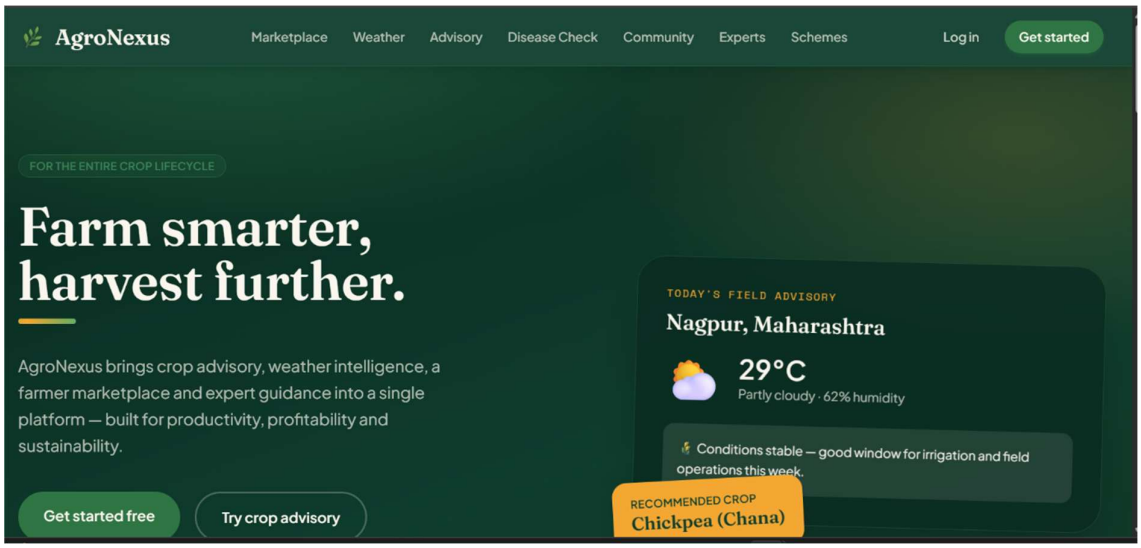



**Figure 9.3: Weather Information Display**

**2. Non-Functional Requirements:**

Non-functional requirements describe **how the system should work**. They focus on the quality, performance, security, usability, and reliability of the application.

**a) Performance:**

The system should provide weather information within a **short amount of time** after the user searches for a city. The API request and response should be handled efficiently so that the user does not have to wait for a long time.

**b) Security:**

The **API key** used to access the weather service should be kept secure. It should not be unnecessarily displayed on the user interface or shared with users. Proper methods should be used to protect sensitive information.

**c) User Interface (UI):**

The user interface should be **simple, clean, and attractive**. The search box, search button, temperature, humidity, wind speed, and weather condition should be properly arranged and clearly visible.

**d) User Experience (UX):**

The system should be easy to use for all users. A user should not need technical knowledge to operate the application. The user should simply enter a city name, click the search button, and view the weather information.

**e) Reliability:**

The system should work properly when the Weather API and internet connection are available. If there is an API or network problem, the system should show an appropriate message instead of stopping unexpectedly.

**f) Usability:**

The application should be easy to understand and operate. Buttons, labels, messages, and weather information should be displayed in a clear manner.

**g) Compatibility:**

The system should work properly on commonly used **web browsers and different screen sizes**. The UI should be designed so that users can access the application comfortably.

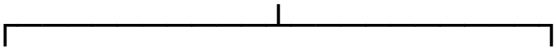
**h) Maintainability:**

The source code should be properly organized so that future changes can be made easily. New features such as **5-day weather forecast, weather alerts, weather icons, and location detection** can be added in the future.

**i) Accuracy:**

The system should correctly display the weather data received from the API. The temperature, humidity, wind speed, and weather condition should be shown without incorrect modification of the API data.

**Requirement Analysis**



| **Functional Requirements** | **Non-functional Requirements** |
|---|---|
| • City Search | • Performance |
| • API Connection | • Security |
| • Temperature | • UI/UX |
| • Humidity | • Reliability |
| • Wind Speed | • Usability |
| • Weather Condition | • Compatibility |
| • Error Handling | • Maintainability |

## Software Requirement Specification (SRS):

Software Requirement Specification (**SRS**) is a document that describes the requirements and features of the **Weather Forecast System Using API**. It explains what the system will do, what modules are required, and what software and hardware are needed.

### 1. Purpose:

The main purpose of this system is to provide **current weather information** of a selected city. The user enters a city name, and the system uses a Weather API to fetch and display weather details.

### 2. Main Features:

The main features of the system are:

- **City Search:** User can enter the name of a city.
- **Weather API:** System connects with the API to get weather data.
- **Temperature:** Displays current temperature.
- **Weather Condition:** Shows conditions such as Sunny, Cloudy, Rainy, etc.
- **Humidity:** Displays humidity percentage.
- **Wind Speed:** Displays current wind speed.
- **Error Handling:** Shows an error message for an invalid city or API.
- **User-Friendly UI:** Provides a simple and easy-to-use interface.

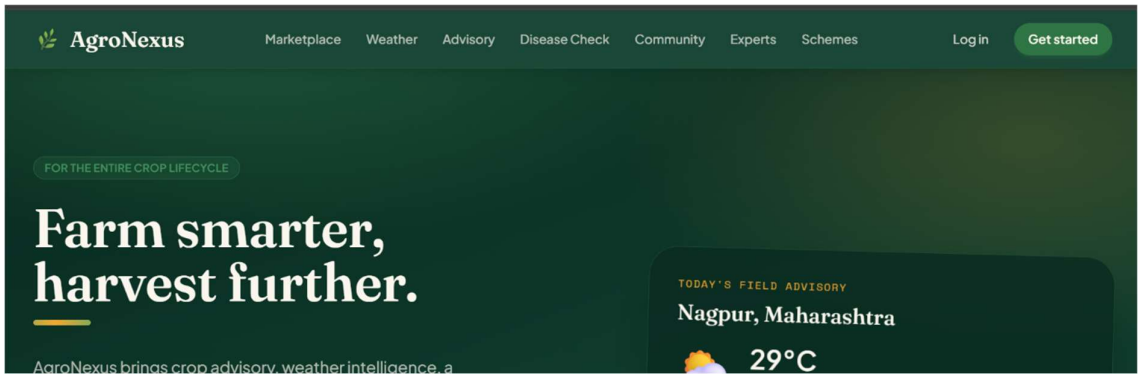



**Figure 9.5: Main Interface of Weather Forecast System**

**3. System Modules:**

The system is divided into the following modules:

**1. User Interface Module:**

Allows the user to enter a city name and displays the weather information.

**2. API Request Module:**

Sends the city name to the Weather API and receives the weather data.

**3. Data Processing Module:**

Processes the API response and extracts required information such as temperature, humidity, and wind speed.

**4. Weather Display Module:**

Displays the processed weather information clearly on the screen.

**5. Error Handling Module:**

Handles invalid city names, API errors, and network problems.

User Input

↓

API Request

↓

Weather API

↓

Data Processing

↓

Weather Result

## 4. Input and Output

### Input:

The main input is the **city name** entered by the user.

### Output:

The system displays:

- City Name
- Temperature
- Weather Condition
- Humidity
- Wind Speed

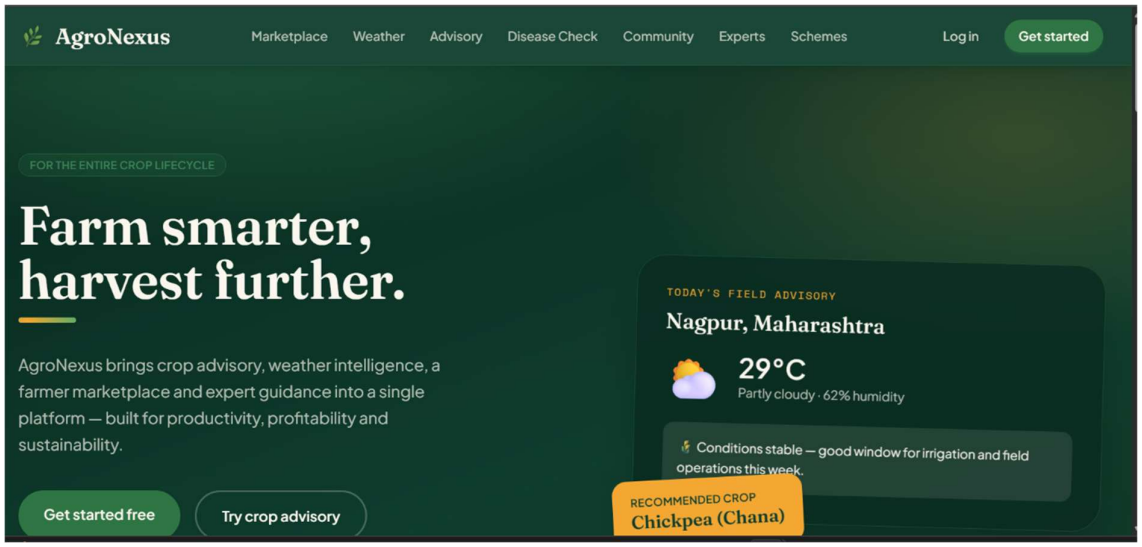



**Figure 9.7: Weather Information Display**

## 5. Software and Hardware Requirements

### Software Requirements:

Python, HTML, CSS, JavaScript, Weather API, Visual Studio Code, Web Browser, Internet Connection

### Hardware Requirements:

Computer/Laptop, Minimum 4 GB RAM, Basic Processor, Internet Connection

## System Design:

### Architecture Diagram:

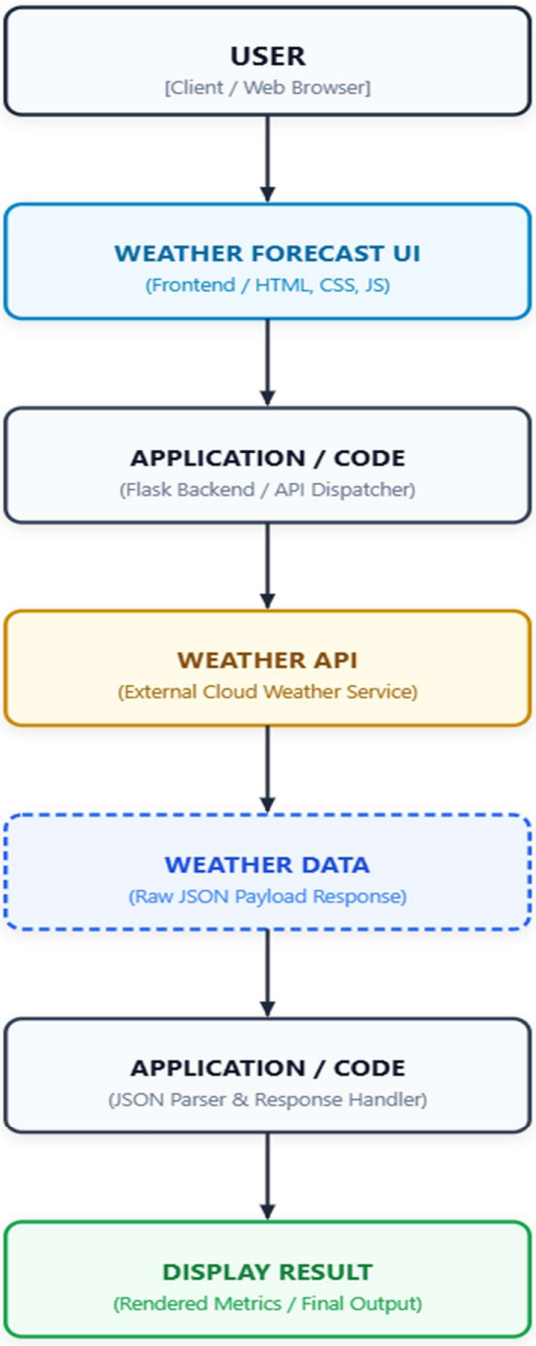



Figure: System Architecture of Weather Forecast System

**Data Flow Diagram (DFD):**

**DFD Level 0:**

A Data Flow Diagram (DFD)—also called a context diagram—is a high-level visual model that represents an entire software system as a single central process interacting with external entities

- **Single Process Node:** The entire software system is shown as one core bubble or process (usually numbered 0).
- **External Entities:** It displays outside actors like users, organizations, or external hardware/software systems that send data to or receive data from the system.
- **Data Flows:** It uses labeled arrows to map the primary inputs entering the system and the outputs leaving it.
- **No Internal Details:** Internal databases (data stores), internal subprocesses, and complex control logic are omitted at this level

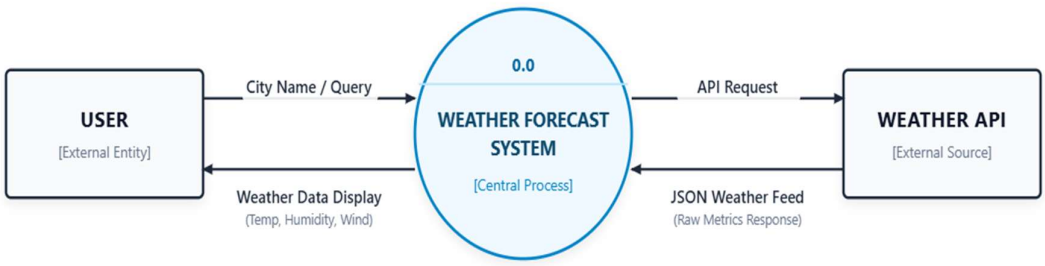



Figure: Level 0 DFD of Weather Forecast System

**DFD Level 1:**

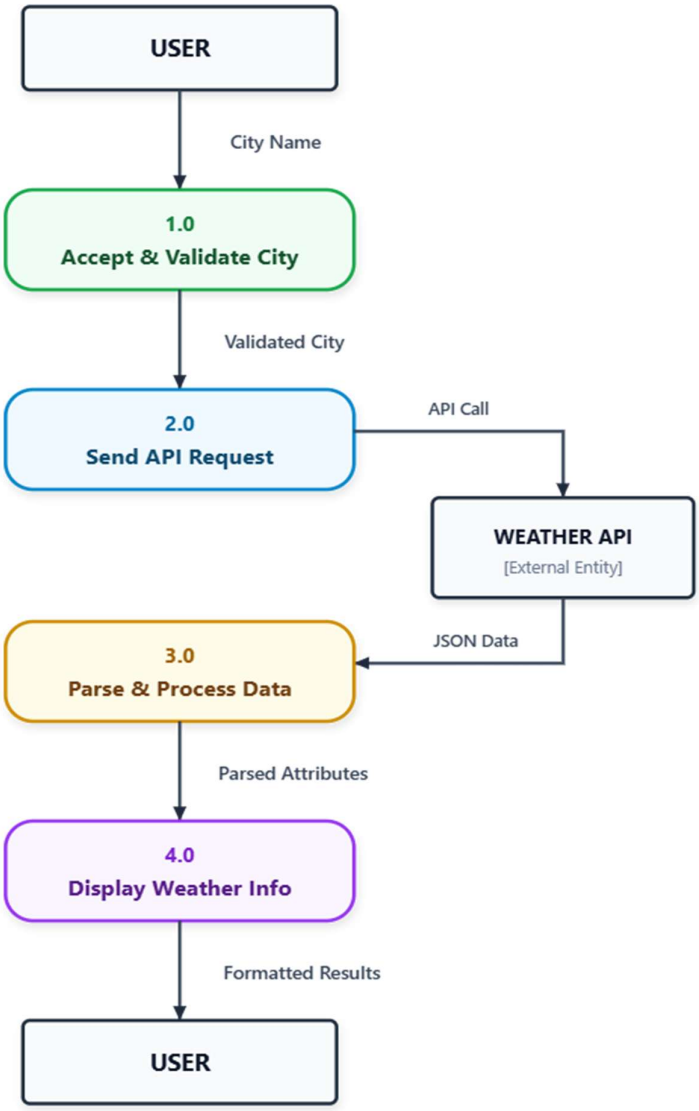



Figure: Level 1 DFD of Weather Forecast System

**UML Diagrams**

**Use Case:**

- Defines Scope: It clearly establishes the boundary line between what is inside the software system and what remains outside. [1, 2]
- Audience Friendly: Its extreme simplicity makes it easy for non-technical stakeholders, managers, and clients to grasp at a single glance. [1, 2, 3]
- Foundation for Expansion: It serves as the parent diagram that gets decomposed into more detailed Level 1 and Level 2 sub-processes later in design.

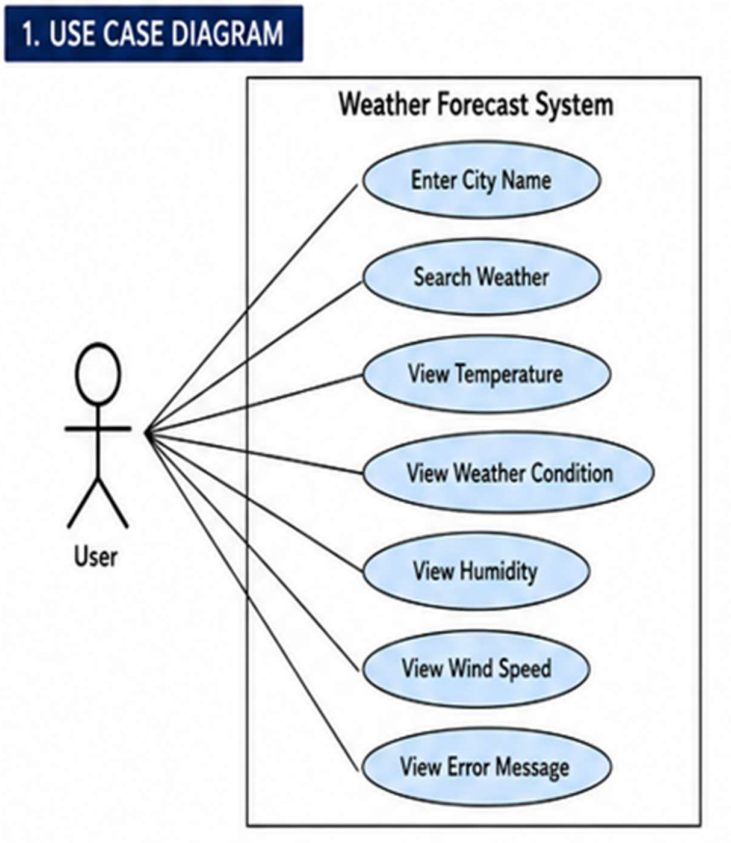



Figure: Use Case Diagram of Weather Forecast System

**Class Diagram:**

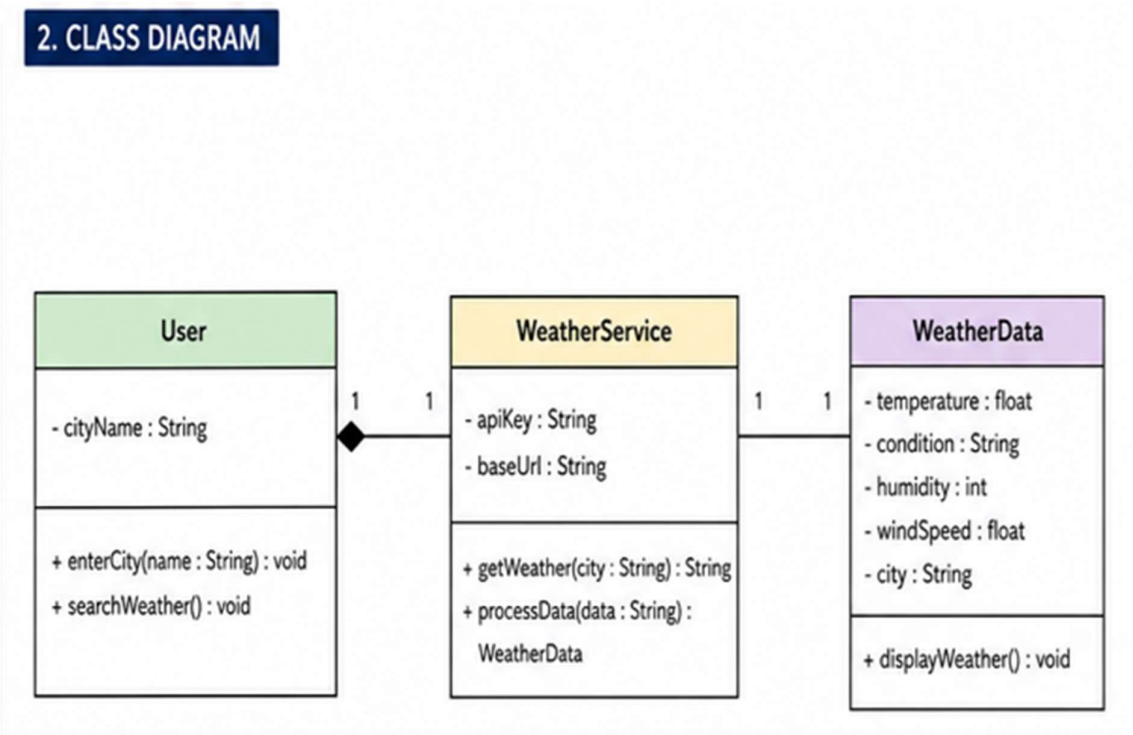



Figure: Class Diagram of Weather Forecast System

Core Components

- Process: Represented by a circle or rounded rectangle in the middle labeled with the system's main function.
- External Entities: Represented by rectangles placed at the outer edges of the diagram.
- Data Flow: Represented by directional arrows showing the movement of information between entities and the central process.

**Sequence Diagram:**

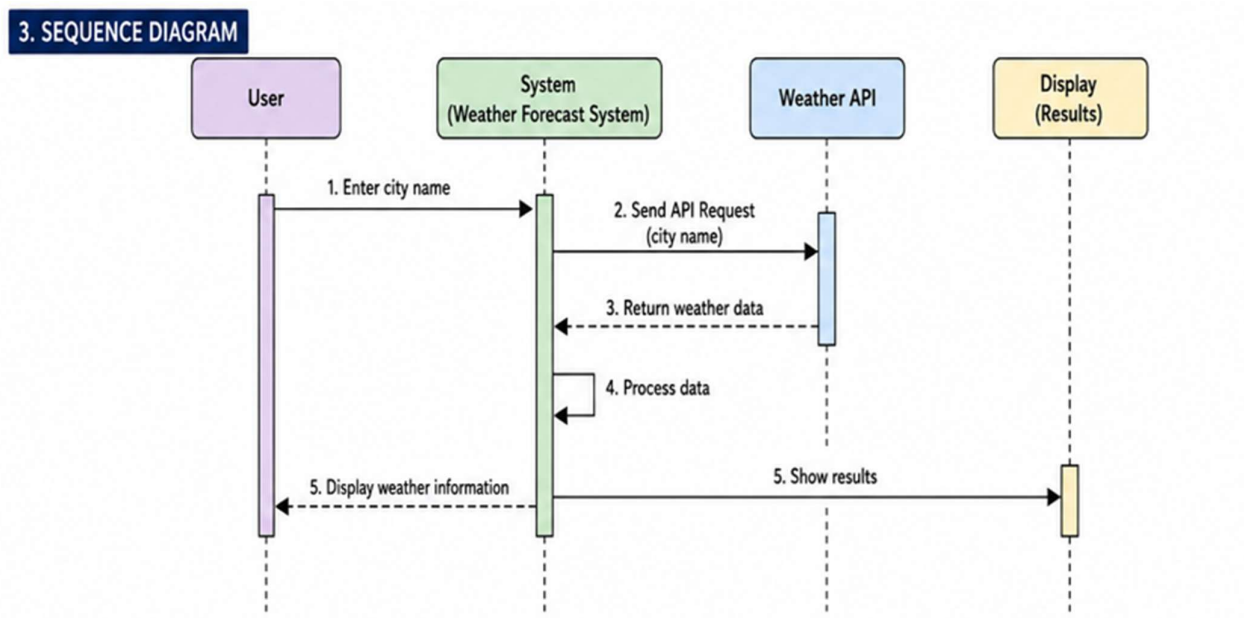



Figure: Sequence Diagram of Weather Forecast System

- Single Process Node: The entire software system is compressed into one central bubble or rounded rectangle (e.g., "Online Bookstore System").

Intellipaat +1

- External Entities Only: It maps out the external sources and destinations of data, such as users, organizations, or other separate software applications.

GeeksforGeeks +1

- High Abstraction: It focuses purely on *what* data enters and leaves the system boundaries rather than *how* the data is processed internally.

Raja Narendra Lal Khan Women's College +1

- No Internal Data Stores: Standard Level 0 diagrams intentionally omit internal databases, file architectures, or local storage repositories, treating the system as a "black box"

## Technology Stack Used:

The **Weather Forecast System Using API** uses different technologies for developing the application and obtaining weather information. The main technology stack is given below.

### 1. Programming Languages

- **Python:** Used for the main application logic and API handling.
- **HTML:** Used to create the structure of the web page.
- **CSS:** Used for designing and styling the user interface.
- **JavaScript:** Used for interactive features and user actions.

### 2. Framework

- **Flask:** A lightweight Python web framework used to develop the web application and connect the frontend with the backend.

### 3. Database

- **SQLite:** Used to store basic application data, if required.

### 4. API

- **Weather API:** Used to fetch real-time weather information such as temperature, humidity, wind speed, and weather condition.

**Technology Stack Summary:**

| Category | Technology |
|---|---|
| Programming Languages | Python, HTML, CSS, JavaScript |
| Framework | Flask |
| Database | SQLite |
| API | Weather API |
| Code Editor | Visual Studio Code |

## Database Design:

Database Design defines how the data is stored and managed in the system. In the **Weather Forecast System Using API**, a database is not required for fetching live weather information because the data is obtained directly from the Weather API.

If the project stores **user search history**, a simple database table can be used to store searched cities and weather details.

**Database Schema:**

**Table: Weather_Search:**

| Field Name | Data Type | Description |
|---|---|---|
| search_id | INTEGER | Unique ID of each search |
| city_name | VARCHAR | Name of the searched city |
| temperature | FLOAT | Temperature of the city |
| humidity | INTEGER | Humidity percentage |
| wind_speed | FLOAT | Wind speed |
| weather_condition | VARCHAR | Current weather condition |
| search_date | DATE | Date of the search |

**ER-Diagram:**

| WEATHER_SEARCH |
|---|
| PK search_id |
| city_name |
| temperature |
| Humidity |
| wind_speed |
| weather_condition |
| search_date |

## Testing:

Testing is an important phase of software development. It is used to check whether the **Weather Forecast System Using API** is working correctly or not. Testing helps to find errors and make sure that the system provides correct weather information to the user.

### Unit Testing:

**Unit Testing** means testing individual parts or functions of the system separately. In our project, different functions such as city input, API request, weather data processing, and error handling can be tested individually.

For example, we can check whether the system correctly accepts a city name and whether the API request returns the required weather data.

### Functional Testing:

**Functional Testing** checks whether the complete system performs its required functions correctly.

For example, the user enters a city name, clicks the search button, and the system should fetch and display the correct weather information.

| Test Case | Input | Expected Result | Status |
|---|---|---|---|
| TC01 | Mumbai | Mumbai weather information is displayed | Pass |
| TC02 | Pune | Pune weather information is displayed | Pass |
| TC03 | Invalid city name | Error message is displayed | Pass |

| | | | |
|---|---|---|---|
| TC04 | Empty input | User is asked to enter a city name | Pass |
| TC05 | Valid city + API request | Temperature and weather details are displayed | Pass |
| TC06 | Internet/API unavailable | Proper error message is displayed | Pass |

## Limitations:

Every software system has some limitations. The **Weather Forecast System Using API** also has some limitations because it is a small project with limited features.

The main limitations are:

1. **Internet Dependency:** The system requires an active internet connection to get weather data from the API.
2. **API Dependency:** The system depends on the Weather API. If the API is unavailable or stops working, weather information may not be displayed.
3. **Limited Forecast:** The current version mainly provides current weather information. A detailed **5-day or 7-day forecast** is not implemented.
4. **Limited Weather Information:** Only basic details such as temperature, humidity, wind speed, and weather condition are displayed.
5. **Location Detection:** Automatic detection of the user's current location is not implemented.
6. **Weather Alerts:** Features such as heavy rain, storm, or extreme temperature alerts are not available.
7. **Database Features:** User accounts and personal weather search history may not be implemented in the current version.

8. **Future Improvements:** The system can be improved by adding **weather maps, multiple-day forecasts, weather alerts, automatic location detection, and a mobile-friendly interface**.

## Future Enhancements:

The **Weather Forecast System Using API** can be improved in the future by adding more features and making the system more useful and user-friendly.

The following features can be added in future:

1. **5-Day / 7-Day Forecast:** The system can provide weather predictions for the next 5 or 7 days.
2. **Automatic Location Detection:** The system can automatically detect the user's current location and show its weather information.
3. **Weather Alerts:** Notifications can be added for heavy rain, storms, high temperature, and other severe weather conditions.
4. **Weather Maps:** Interactive maps can be added to show weather conditions of different locations.
5. **More Weather Details:** Additional information such as **visibility, pressure, sunrise, sunset, and UV index** can be displayed.
6. **Temperature Unit Selection:** Users can select temperature units such as **Celsius (°C) or Fahrenheit (°F)**.
7. **Search History:** A database can be added to save the user's previously searched cities.
8. **Mobile-Friendly Design:** The system can be improved to work properly on mobile phones, tablets, and computers.
9. **Weather Icons and Charts:** Weather icons and graphical charts can be added to make the information easier to understand.
10. **AI-Based Weather Assistant:** An AI chatbot can be added to answer weather-related questions and provide simple recommendations based on weather conditions.

# CONCLUSION

The Weather Forecast System Using API was successfully developed to provide weather information in a simple and user-friendly way. The system uses a Weather API to fetch information such as temperature, humidity, wind speed, and weather condition for a selected city.

Through this project, we learned how to develop a web-based application, work with APIs, send and receive data, process API responses, design a user interface, and perform software testing. We also learned how Software Engineering concepts such as SDLC, Requirement Analysis, SRS, UML diagrams, and Testing are applied in a real project.

The project met its main objectives. It allows users to search for a city and get its current weather information. The system provides the required weather details and handles basic invalid inputs and API errors. Overall, this project helped us understand the practical use of API integration and software development concepts.

# REFERENCES

The following resources, websites, and tools were referred to during the development of the **Weather Forecast System Using API**:


1. **Open Weather API Documentation** – Used for understanding API requests and weather data.
   https://openweathermap.org
2. **Python Documentation** – Used for Python programming and API handling https://docs.python.org/
3. **Flask Documentation** – Used for developing the web application using Flask. https://flask.palletsprojects.com/
4. **MDN Web Docs** – Used for HTML, CSS, and JavaScript concepts. https://developer.mozilla.org
5. **Visual Studio Code** – Used as the code editor for developing the project. https://code.visualstudio.com/
6. **Google Chrome** – Used for running and testing the web application. https://www.google.com